TripEase: An Integrated Trip Planning and Itinerary Management System

Project Proposal for UCS503P
Submitted to: Ms. Paramveer Sidhu
Thapar Institute of Engineering and Technology

Manraj Singh, COE[*] Vatsal Chauhan, COE[†] Nimitt Malhotra, COE[‡]

August 25, 2026

1 Higher-order goal

. . . secondary framing

Planning a shared trip is a coordination problem before it is a travel problem. A group agrees on a destination, then spends days re-deciding the same questions — what are we seeing on Tuesday, who paid for the cab, is the fort before or after lunch — because the answers live in five different apps and nobody owns the current version.

The broader goal we are framing this project against is *reducing coordination overhead in small-group decision making*. That is deliberately larger than one web application, and we are not claiming to solve it. The immediate engineering objective is narrower and testable: give a small group one shared, editable artefact — a trip — that holds the plan, the geography and the money together, and measure whether producing that artefact takes materially less effort than the current do-it-yourself workflow.

A modest secondary benefit is that a plan with real route information attached tends to be a less wasteful plan. Groups that can see their day laid out on a map avoid the backtracking that comes from ordering stops by memory. We mention this as framing only; it is not something we intend to measure in this course.

2 Time-to-value

. . . primary heuristic

- The first shippable increment is a *thin vertical slice*: sign in, create a trip, add named places to it, see it again on reload. It touches authentication, the data model, the UI shell and the deployment pipeline, so it de-risks all four at once instead of building any one of them to completion.
- We work in one-week iterations, with a demonstrable increment deployed to a staging URL at the end of each. Feedback is collected from real classmates planning real trips, not from ourselves.
- CI is set up in iteration 1, before the feature work grows. Every push runs lint, unit tests and a production build; merges to `master` deploy to staging automatically. The cost of doing this early is roughly a day; the cost of retrofitting it in week six is much higher.
- We prefer a narrow feature that is deployed over a broad feature that is half-written on a branch. Where the two conflict, scope is cut, not the release.

[*]Roll No: 1024030712, Email: msingh17_be24@thapar.edu
[†]Roll No: 1024030714, Email: vchauhan_be24@thapar.edu
[‡]Roll No: 1024030709, Email: nmalhotra_be24@thapar.edu

3 Problem Statement

Trip planning is not blocked by a lack of tools. It is blocked by the fact that the tools do not share state. A student group at this institute planning a four-day trip to Jaipur or Manali will typically end up using: a search engine and travel blogs for deciding what is worth seeing; Google Maps for checking whether two places are anywhere near each other; a notes app or a shared document for the day-by-day plan; a spreadsheet for the budget and for who owes whom; and a WhatsApp group where all of the above get pasted, discussed and then lost in the scrollback.

The observable pain points are:

- **No single source of truth.** The itinerary exists in three places at once and they disagree. The most recent version is whichever one the loudest person is reading from.
- **Geography is disconnected from the plan.** The day order is decided in a text document, where nothing warns you that stop 2 and stop 3 are on opposite sides of the city. The mismatch is discovered on the road.
- **Manual, repeated arithmetic.** Budgets are re-totalled by hand every time something changes, and expense splitting is done after the trip from memory and screenshots.
- **Sharing is lossy.** Sending the plan to someone means exporting it into a message, after which their copy immediately goes stale.
- **Re-work.** None of the effort is reusable. The next trip starts from an empty document again.

Contextual relevance. The stakeholders here are our immediate peers. Group trips planned over a weekend by five or six students, on mid-range Android phones, on hostel Wi-Fi, are a workload we can observe directly and get feedback on within a single iteration. We are not designing for a travel agency.

Formal statement. *Design and build a web-based system that lets a user assemble a multi-day trip — destinations, an ordered day-wise itinerary, the route between stops, and a categorised budget with actual expenses — as a single persistent shared record, and that reduces the time and the number of separate tools required to produce a complete, agreed itinerary.*

4 Proposed Solution

. . . Software Proposition

4.1 Overview

TripEase is a responsive web application in which a *trip* is the central object and everything else hangs off it.

- **Accounts and trips:** email/password and Google sign-in; a dashboard listing the user's trips; create, rename, edit dates and delete.
- **Place capture:** search for a place by name using an autocomplete backed by a maps provider, and attach it to the trip with its coordinates and identifier resolved.
- **Day-wise itinerary:** assign captured places to days and order them within a day. Reordering is a drag or an up/down control, not a re-typed document.
- **Map and route view:** the selected day's stops drawn as ordered markers with a route polyline, plus leg distance and travel time, so the plan is checked against geography while it is being made rather than afterwards.

- **Budget and expenses:** a planned budget split by category, actual expenses logged against those categories, and a running planned-versus-actual total.
- **Sharing:** a read-only link to the current itinerary, so the shared copy stays live instead of being a stale paste.

4.2 Core workflow

1. User signs in and creates a trip with a destination and a date range. The number of days follows from the dates.
2. User searches for places and adds them to the trip's place list.
3. User assigns places to days and orders them.
4. The map view draws the day's route and reports leg distances and durations; the user adjusts the order if it looks wrong.
5. User sets a budget per category and logs expenses as they occur.
6. User shares a read-only link with the rest of the group.

Figure 1 shows this as a flow. Every step after sign-in writes to the same trip document, so there is only ever one current version of the plan.

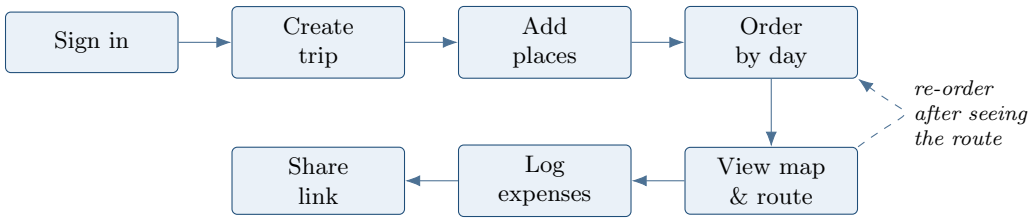

Figure 1: Proposed core workflow. The dashed edge is the feedback loop that the current do-it-yourself workflow lacks: the plan is checked against real geography while it is being written.

4.3 What this is not

To keep the scope honest: TripEase does not book anything, does not price flights or hotels, and does not generate itineraries automatically. Those are discussed in §9 as later or out-of-course work. The first release is a planning and record-keeping tool.

4.4 Operational constraints

- Mobile-first responsive layout. Planning happens on phones; the map view must be usable on a small screen, not merely render on one.
- Tolerant of poor connectivity: reads are cached where the client library allows it, and a failed map call degrades to the list view rather than blanking the page.
- No dependence on unpublished or research-grade algorithms. Ordering aids are heuristic and always overridable by the user.

5 Solution Approach

. . . Engineering focus

This is an engineering course project. The interesting work is in the data model, the access rules, the integration boundaries and the release process — not in inventing new algorithms.

5.1 Architecture

. . . web-based solution

We propose a client-centric architecture over managed backend services rather than a hand-written server tier, as shown in Figure 2.

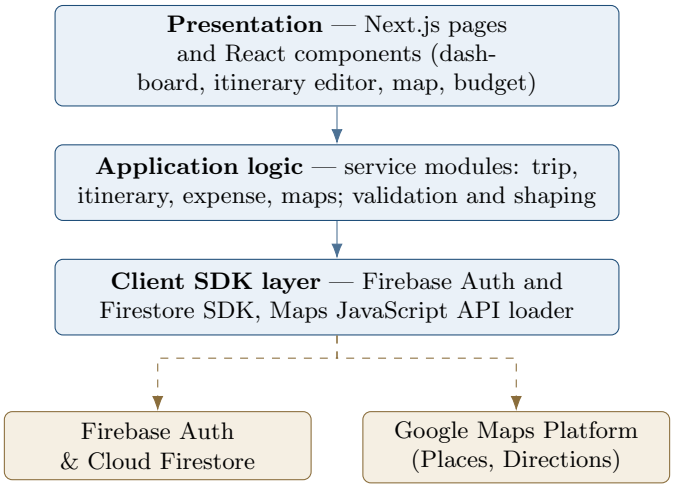

Figure 2: Proposed architecture. Solid edges are in-application calls; dashed edges cross a trust boundary into managed third-party services.

Why no separate backend server? A conventional three-tier design would mean writing, hosting, securing and monitoring an API server whose only real job would be to forward CRUD operations to a database. For a three-person team on a one-semester schedule that is overhead without a corresponding benefit. Firebase Authentication and Firestore provide identity and persistence directly, and Firestore security rules move authorisation into a declarative, testable artefact that lives in the repository. The trade-off we accept is vendor lock-in and less control over query planning; we consider that acceptable at this scale, and the service modules in the application layer exist specifically so that data access is not scattered through the components if it ever has to change.

We do expect to need a small amount of server-side code, and Next.js route handlers cover it without a separate deployment: the Directions call and the read-only share endpoint should run server-side so the corresponding API key is never shipped to the browser.

5.2 Maps integration

. . . planned, not connected

Three distinct capabilities are involved, and it is worth separating them because they have different cost and caching characteristics:

- **Places / Autocomplete** — turning what the user typed into a specific place with a stable identifier and coordinates.
- **Geocoding** — address or place name to latitude/longitude, and the reverse where a user drops a pin.
- **Directions** — an ordered list of stops to a route, with per-leg distance and duration.

Maps Platform terms restrict how long most place content may be stored, while place identifiers may be retained. Our data model therefore stores the identifier, the coordinates and a user-visible label, and re-fetches presentation details on demand. This is a real constraint on the schema, not a footnote, and we have designed around it rather than discovering it later.

5.3 Ordering aid *... non-research*

Suggesting a sensible order for a day's stops is a travelling-salesman variant. We are explicitly not solving it. For the five-to-eight stops a real day contains, we apply a nearest-neighbour pass over straight-line distances and present it as a suggestion the user may accept or ignore. The user's manual order always wins, and the suggestion is never applied silently. We will not claim optimality.

5.4 CI/CD

- GitHub Actions runs lint, unit tests and a production build on every push and pull request.
- Firestore security rules are tested in CI against the emulator, so an authorisation regression fails the build rather than reaching staging.
- Merges to `master` deploy to a staging environment with its own Firebase project, so pilot data never mixes with development data.
- Secrets live in repository secrets and in `.env.local`; `.env.example` is committed, real keys are not.

6 Evaluation Criterion *... measurable and attributable*

All figures below are **pre-registered targets** for the pilot, fixed before development so that they cannot be adjusted to fit whatever we happen to observe. They are not results. Nothing has been measured yet.

6.1 Primary metric

Time-to-complete-itinerary (TTCI): elapsed time from trip creation to the trip reaching a *complete* state, defined as every day in the range having at least one ordered stop and a budget figure recorded.

- **Target:** median TTCI $\leq$ 15 minutes for a standardised task — a 3-day trip with at least 5 stops — for first-time users.
- **Attribution:** computed from server timestamps written by the application (`createdAt` on the trip document, and the write that first satisfies the completeness predicate). It is derived from data the system already stores, so it does not depend on self-reporting.
- **Baseline:** the same task performed with the participant's existing tools, timed by observation, collected from the same participants before they use TripEase.

6.2 Secondary metrics

- **Tool count:** number of distinct applications the participant used to complete the task, before versus after. Target: reduced to one for the planning task itself.
- **Task completion rate:** share of participants who reach a complete itinerary unaided. Target: $\geq$ 80%.
- **Re-order rate after route view:** how often a user changes a day's order after seeing the route. This is our check on whether the map is actually informing the plan or is decoration.
- **Perceived clarity:** single 1–5 item on whether the itinerary was easy to follow. Target: mean $\geq$ 4.

- **Reliability:** sign-in and trip-save available on staging during the pilot window. Target: $\geq$ 99%.
- **Delivery cadence:** number of iterations that ended with a green build deployed to staging. Target: every iteration. This one measures our process, not the product, and the course criteria make it a first-class metric.

6.3 Pilot plan *... high-level*

- 12–20 participants drawn from classmates who have planned a group trip in the last year.
- Each performs the standardised task once on their existing tools (baseline, observed) and once on TripEase, with order counterbalanced across participants to blunt the learning effect.
- Instrumentation is in the application from iteration 2, so the metric is available for every iteration and not just at the end.
- Participants are told what is being recorded; no location history or contact data is collected.

7 Scalability *... with foresight*

1. **Scalable in principle.** Growth here is in trips and concurrent editors, not in compute.
   - Trips are a top-level collection keyed by owner, with itinerary items and expenses as subcollections. Queries stay bounded to a single trip; a user with 200 trips loads a page of them, not all of them.
   - Every list query is indexed and paginated with cursors, so cost is proportional to what is displayed rather than to collection size.
   - Counters and totals that would otherwise require reading a whole subcollection are maintained as denormalised fields on the trip document, updated in a transaction.
   - The client is stateless with respect to the server; there is no session affinity to preserve.
2. **Operationally deployable.** The deployment story is deliberately boring.
   - Static and server-rendered output hosted on a managed platform with a CDN in front.
   - Managed, replicated datastore; no database administration on our side.
   - Separate development, staging and production projects with separate keys and quotas.
   - Deployment is a pipeline job, so a release is repeatable and revertible rather than a manual sequence someone remembers.

The honest limit is cost, not architecture: Maps Platform calls are billed per request, so at large scale the constraint becomes autocomplete session management and caching what the terms permit, rather than servers.

8 Engine availability heuristic

Every component of the core solution already exists as a mature, documented product. Table 1 lists them with the reason for each choice.

Table 2 contrasts the current do-it-yourself workflow with the proposed one. The point is not that the existing tools are poor — each is better at its own job than anything we will build. The point is that state is not shared between them.

Table 1: Engines selected, with rationale. No component requires research work.

Concern	**Engine**	**Why this one**
UI and routing	Next.js (React)	File-based routing and server route handlers let us keep the Directions key server-side without operating a second service; one deployable artefact.
Identity	Firebase Authentication	Email/password and Google sign-in without hand-rolling password storage or session handling — an area where a student implementation is most likely to be quietly wrong.
Persistence	Cloud Firestore	Document model matches a trip's nested, variable-shape structure; live listeners give multi-device consistency for free, which is the basis for later collaborative editing.
Geography	Google Maps Platform	Places, Geocoding and Directions from one provider with a single billing account and consistent place identifiers across all three.
Authorisation	Firestore security rules	Access control as a declarative file in the repository, testable in CI against the emulator.
Testing	Vitest, React Testing Library, Firebase emulator	Standard, fast, runs headless in CI.
CI/CD	GitHub Actions	Already integrated with the course repository; no additional service to configure.

Table 2: Current workflow versus proposed workflow.

Concern	**Current workflow**	**Proposed (TripEase)**
Research	Search engine, blogs	Unchanged; TripEase does not replace discovery
Place list	Pasted into chat or notes	Structured records attached to the trip
Day plan	Notes app or shared document	Ordered itinerary bound to trip dates
Route check	Maps app, opened separately	Route drawn from the itinerary itself
Budget	Spreadsheet, re-totalled by hand	Categorised budget with running totals
Sharing	Exported paste, immediately stale	Read-only link to the live trip
Source of truth	Ambiguous	The trip document
Reuse	None	Saved trips remain available

9 Project scope and deliverables

. . . iteration-friendly

9.1 Initial deliverable

. . . first iteration

- Sign-up and sign-in; authenticated dashboard.
- Create a trip (destination, dates); list and delete own trips.
- Add named places to a trip and see them persist across reloads.
- Firestore rules restricting every trip to its owner, with rule tests in CI.
- CI: lint, unit tests, production build on every push.
- CD: automatic deploy of `master` to a staging URL.

9.2 Subsequent deliverables

- **Iteration 2:** day-wise itinerary with reordering; TTCI instrumentation.
- **Iteration 3:** map view, markers, route polyline, leg distance and duration.
- **Iteration 4:** budget categories, expense logging, planned-versus-actual view.
- **Iteration 5:** read-only share link; accessibility and mobile pass.
- **Iteration 6:** pilot, measurement against §6, and refinement from what it shows.

Figure 3 shows the same plan on a timeline. It is illustrative — iteration boundaries will move as feedback arrives, which is the point of working this way.

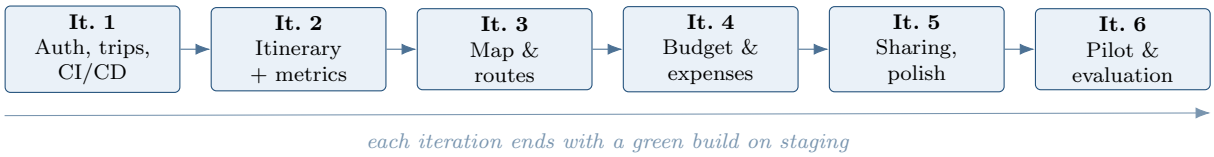

Figure 3: Illustrative iteration roadmap.

9.3 Out of scope for this course

Weather, AI-generated itineraries, collaborative real-time editing, hotel and transport booking, and multi-language support are recorded as future scope. Listing them here is a scope boundary, not a promise.

10 Risks and mitigations

- **Maps quota and billing.** Autocomplete is the expensive call. Mitigation: debounce input, use session tokens, cap per-user calls in development, and keep the itinerary usable without the map so a quota exhaustion degrades one feature instead of the product.
- **Key exposure.** A committed or unrestricted key is the realistic security failure for a project of this kind. Mitigation: keys in environment variables only; the browser key restricted by HTTP referrer and by API; the Directions key used only from server-side route handlers; a secret scan in CI.

- **Authorisation defects.** With no server tier, Firestore rules *are* the authorisation layer. Mitigation: rules are version-controlled and unit-tested against the emulator in CI; the default is deny.
- **Scope creep towards AI features.** Mitigation: the out-of-scope list above is treated as binding for the course; anything on it needs an explicit decision to move.
- **Vendor lock-in.** Mitigation: all data access goes through service modules rather than being called from components, so the surface that would have to change is small and enumerable.
- **Uneven contribution across three members.** Mitigation: work tracked as issues with named owners, review required before merge, and the per-member journals in this repository as the record.

11 Summary

TripEase proposes a web application that keeps a trip's itinerary, geography and budget in one shared record, addressing workflow fragmentation rather than any absence of tools. It fits the course criteria on each heuristic: the stakeholders are our immediate peers and the problem is observable now; every component is a mature, documented engine, with the one genuinely hard sub-problem deliberately reduced to an overridable heuristic; the architecture is deployable on managed services from iteration 1; and success is defined by a measurable, system-attributable metric — time-to-complete-itinerary against an observed baseline — fixed in advance.

Nothing described here has been implemented yet. This document is a proposal, and the numbers in §6 are targets to be tested, not findings.